Лабораторная работа №1 — Знакомство со средствами разработки программ на языке C

Предмет: OS And Syst Software **Курс:** 2026 HR S5 OS And Syst Software **Выполнил:** Aliaksei Ioda **Дата:** 26.10.2026

Цель работы

Подготовить рабочее окружение для разработки на C, разобраться, из каких этапов состоит создание программы, и научиться собирать, запускать и отлаживать программу как из IDE, так и из командной строки.

Окружение

Компонент	Версия / значение
ОС	macOS 12.6 (Darwin 21.6.0, x86_64)
Компилятор	Apple clang 14.0.0 (`clang-1400.0.29.202`)
Отладчик	lldb 14.0.0 (`lldb-1400.0.38.17`)
Сборка	GNU Make 3.81
IDE	Visual Studio Code 1.143 (встроенный терминал)

> **Почему clang/lldb, а не gcc/gdb.** На macOS `gcc` — это псевдоним `clang`, а `gdb` штатно не поставляется; вместо него используется `lldb`. Для этой лабораторной разница только в именах команд: все этапы (препроцессор, компиляция, компоновка, точки останова, Step Over/Step Into) выполняются теми же средствами. Для следующих работ, где нужны `/proc`, `valgrind` и `strace`, потребуется Linux (WSL 2, виртуальная машина или Docker).
>
> Скрипт `scripts/run_lab.sh` автоматически выбирает `gcc + gdb` на Linux и `clang + lldb` на macOS.

На Linux достаточно:

```
sudo apt update
sudo apt install build-essential gdb valgrind strace
bash scripts/run_lab.sh
```

Структура работы

```
lab-1/
├── README.md                — этот отчёт
├── src/
│   ├── hello.c              — минимальная программа
│   ├── sum.c                — сумма двух чисел (с функцией add)
│   ├── noinclude.c          — ошибка: нет #include <stdio.h>
│   └── badcall.c            — ошибка: вызов функции без определения
├── scripts/
│   ├── run_lab.sh           — воспроизводимый прогон всех этапов
│   ├── lldb_script.txt      — команды для lldb (macOS)
│   ├── gdb_script.txt       — команды для gdb (Linux)
│   └── stdin.txt            — входные данные 12 и 30
├── .vscode/
│   ├── tasks.json           — сборка hello и sum (VS Code → Run Build Task)
│   └── launch.json          — отладка sum под lldb (F5)
└── artifacts/
    ├── lab-output.txt       — полный лог выполнения
    └── lab-1-report.pdf     — отчёт в PDF
```

Полный лог: `artifacts/lab-output.txt`.

IDE

В Visual Studio Code открыт корень `lab-1`; сборка и запуск выполняются во встроенном терминале (`Terminal → New Terminal`), конфигурации для Build Task и отладки лежат в `.vscode/`. Расширение C/C++ (`ms-vscode.cpptools`) в этом окружении не установлено — отладка выполнена напрямую в `lldb`, как показано в разделе 5. Это не меняет результат: команды сборки и отладки те же, что использует IDE.

---

1. Проверка окружения

```
$ uname -a
Darwin MacBook-Pro-Aliaksei.local 21.6.0 Darwin Kernel Version 21.6.0 ... x86_64

$ clang --version
Apple clang version 14.0.0 (clang-1400.0.29.202)
Target: x86_64-apple-darwin21.6.0

$ lldb --version
lldb-1400.0.38.17

$ make --version
GNU Make 3.81
```

На Linux вместо этого выводятся:

```
$ gcc --version
gcc (Ubuntu ...) 13.x
$ gdb --version
GNU gdb (Ubuntu ...) 14.x
```

2. Сборка и запуск hello.c из командной строки

```
#include <stdio.h>

int main(void) {
    printf("Hello, OS!\n");
    return 0;
}
```

```
$ clang -g -O0 src/hello.c -o build/hello
$ ./build/hello
Hello, OS!
$ echo $?
0
```

`-g` добавляет отладочную информацию, `-O0` отключает оптимизации, чтобы отладчик показывал ровно те строки и переменные, которые есть в исходнике.

3. Программа «сумма двух чисел»

```
#include <stdio.h>

int add(int a, int b) {
    int result = a + b;
    return result;
}

int main(void) {
    int num1, num2, sum;

    printf("Введите первое число: ");
    scanf("%d", &num1);

    printf("Введите второе число: ");
    scanf("%d", &num2);

    sum = add(num1, num2);
    printf("Сумма: %d\n", sum);
    return 0;
}
```

```
$ clang -g -O0 src/sum.c -o build/sum
$ printf '12\n30\n' | ./build/sum
Введите первое число: Введите второе число: Сумма: 42
```

`scanf("%d", &num1)` читает целое число и записывает его по адресу `num1`; оператор `&` здесь обязателен.

4. Три стадии сборки

Стадия	Команда	Результат
Препроцессор	`clang -E src/hello.c -o build/hello.i`	`hello.i` — исходник после раскрытия `#include` и макросов
Компиляция	`clang -c src/hello.c -o build/hello.o`	`hello.o` — объектный файл с машинным кодом
Компоновка	`clang build/hello.o -o build/hello_linked`	`hello_linked` — готовый исполняемый файл

```
$ wc -l src/hello.c build/hello.i
       6 src/hello.c
     556 build/hello.i
     562 total

$ file build/hello.o build/hello
build/hello.o: Mach-O 64-bit object x86_64
build/hello:   Mach-O 64-bit executable x86_64
```

Файл после препроцессора в ~93 раза больше исходного: в него целиком подставлено содержимое `stdio.h` и всех заголовков, которые он тянет за собой. Объектный файл (`hello.o`) ещё не является программой: в нём есть машинный код и таблица символов, но адреса функций и данных не разрешены. Компоновщик соединяет объектные файлы с библиотеками, подставляет адреса и создаёт исполняемый файл.

5. Отладка: точка останова, Step Over и Step Into

Отладочный сценарий для lldb:

```
breakpoint set --name main
process launch -i scripts/stdin.txt
next            # Step Over
next
frame variable num1
next
next
frame variable num2
step            # Step Into: заходим в add()
frame variable a b
next
frame variable result
next
frame variable num1 num2 sum
next
frame variable num1 num2 sum
continue
```

Ключевые фрагменты вывода:

```
(lldb) breakpoint set --name main
Breakpoint 1: where = sum`main + 15 at sum.c:11:5

(lldb) process launch -i scripts/stdin.txt
Process stopped
* thread #1, stop reason = breakpoint 1.1
    frame #0: sum`main at sum.c:11:5
-> 11     printf("Введите первое число: ");

(lldb) next
-> 12     scanf("%d", &num1);
(lldb) next
-> 14     printf("Введите второе число: ");
(lldb) frame variable num1
(int) num1 = 12

(lldb) next
-> 15     scanf("%d", &num2);
(lldb) next
-> 17     sum = add(num1, num2);
(lldb) frame variable num2
(int) num2 = 30

(lldb) step
* thread #1, stop reason = step in
    frame #0: sum`add(a=12, b=30) at sum.c:4:18
-> 4      int result = a + b;
(lldb) frame variable a b
(int) a = 12
(int) b = 30

(lldb) next
-> 5      return result;
(lldb) frame variable result
(int) result = 42

(lldb) next
-> 17     sum = add(num1, num2);
(lldb) frame variable num1 num2 sum
(int) num1 = 12
(int) num2 = 30
(int) sum = 37          # присваивание ещё не завершено

(lldb) next
-> 18     printf("Сумма: %d\n", sum);
(lldb) frame variable num1 num2 sum
(int) num1 = 12
(int) num2 = 30
```

```
(int) sum = 42

(lldb) next
Введите первое число: Введите второе число: Сумма: 42
Process exited with status = 0
```

`next` (Step Over) выполняет строку целиком, не заходя внутрь вызванной функции. `step` (Step Into) заходит в `add()` и позволяет увидеть её параметры `a`, `b` и локальную переменную `result`. Значение `sum = 37` на строке 17 — это ещё не результат сложения: присваивание завершается только после возврата из `add()`, и на следующем шаге `sum` становится `42`.

6. Намеренная ошибка: отсутствует `#include <stdio.h>`

```
int main(void) {
    printf("Hello, OS!\n");
    return 0;
}
```

```
$ clang -g -O0 src/noinclude.c -o build/noinclude
src/noinclude.c:2:5: error: implicitly declaring library function 'printf'
      with type 'int (const char *, ...)' [-Werror,-Wimplicit-function-declaration]
    printf("Hello, OS!\n");
    ^
src/noinclude.c:2:5: note: include the header <stdio.h> or explicitly provide
      a declaration for 'printf'
1 error generated.
```

Ошибку выдаёт **компилятор** на стадии компиляции: без объявления из `stdio.h` он не знает, что `printf` существует и какие у неё параметры. В зависимости от версии компилятора и флагов это может быть предупреждением (`-Wimplicit-function-declaration`) или ошибкой; здесь clang считает это ошибкой.

7. Намеренная ошибка: вызов функции без определения

```
#include <stdio.h>

int does_not_exist(void);   /* объявление есть, определения нет */

int main(void) {
    printf("before\n");
    does_not_exist();
    printf("after\n");
    return 0;
}
```

```
$ clang -g -O0 src/badcall.c -o build/badcall
Undefined symbols for architecture x86_64:
  "_does_not_exist", referenced from:
      _main in badcall-8cb5cf.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1
```

Здесь компиляция проходит: объявление функции есть, поэтому компилятор проверил вызов и сгенерировал объектный файл. Ошибка `undefined reference` (в macOS — `Undefined symbols`) возникает на **стадии компоновки**, когда компоновщик не находит тело функции ни в одном объектном файле или библиотеке.

Контрольные вопросы

1. Из каких шагов состоит создание программы? Что такое IDE?

1. Набор текста в редакторе и сохранение в файл (`.c`).
2. Компиляция — преобразование исходника в машинный код.
3. Запуск программы.
4. При ошибках — отладка и повторение цикла с шага 1.

IDE (Integrated Development Environment) объединяет в одной оболочке текстовый редактор, компилятор, компоновщик, отладчик и средства сборки проекта, поэтому цикл «правка → сборка → запуск → отладка» не требует переключения между отдельными программами.

2. Какие три стадии выполняет компилятор? Чем компиляция отличается от компоновки?

- **Препроцессор** раскрывает `#include`, `#define` и условную компиляцию.
- **Компиляция** переводит C-код в машинные инструкции и создаёт объектный файл.
- **Компоновка (линковка)** объединяет объектные файлы и библиотеки, разрешает внешние символы и создаёт исполняемый файл.

Компиляция работает с одним исходным файлом и ещё не знает, где находится тело внешней функции. Компоновка как раз связывает вызовы с их определениями в других объектных файлах и библиотеках.

3. Что такое объектный файл? Почему исполняемый файл не запустится на другой платформе?

Объектный файл — результат компиляции одного исходного файла: машинный код, данные, таблица символов и информация для перемещения (relocation). Он ещё не готов к запуску, потому что адреса внешних функций и данных не разрешены.

Исполняемый файл зависит от архитектуры процессора (x86_64, ARM64) и операционной системы: формата файла (ELF в Linux, Mach-O в macOS, PE в Windows), системных вызовов и ABI. Поэтому программу, собранную под одну платформу, нельзя просто запустить на другой — нужна пересборка.

4. Что такое точка останова? Чем отличаются Step Into и Step Over?

Точка останова (breakpoint) — место в программе, на котором отладчик приостанавливает выполнение, чтобы можно было посмотреть значения переменных и состояние стека.

- **Step Over** выполняет текущую строку целиком; если в ней есть вызов функции, отладчик не заходит внутрь.
- **Step Into** заходит внутрь вызванной функции и останавливается на её первой строке.

5. Зачем нужны ключи `-g` и `-O0`?

- `-g` добавляет в объектный файл отладочную информацию: имена переменных, номера строк, связь машинного кода с исходником. Без неё отладчик покажет только адреса и ассемблер.
- `-O0` отключает оптимизации. При оптимизации компилятор может переставлять строки, выбрасывать переменные и объединять вычисления, и тогда поведение в отладчике перестаёт совпадать с исходным текстом.

6. Чем WSL отличается от виртуальной машины? Почему онлайн-компилятора недостаточно?

WSL 2 — это настоящее ядро Linux, запущенное в лёгкой виртуальной машине внутри Windows, с прямой интеграцией с файловой системой и терминалом Windows. Полноценная виртуальная машина (VirtualBox, VMware) эмулирует целый компьютер: у неё собственное ядро, диск, сеть и, как правило, графическое окружение; она тяжелее, но и ведёт себя как отдельная машина.

Онлайн-компилятора недостаточно, потому что в курсе нужны второй терминал, файловая система Linux, `/proc`, `valgrind`, `strace` и полноценный `gdb`. Онлайн-сервис даёт только «вставить код → получить вывод» и не позволяет исследовать процессы, память и системные вызовы.

Выводы

В ходе работы было подготовлено окружение для разработки на C: компилятор, отладчик, система сборки и IDE. На практике проверены все три стадии сборки: препроцессор увеличил `hello.c` с 6 до 556 строк, компиляция создала объектный файл, а компоновка — исполняемый. Программа «сумма двух чисел» собрана и запущена из командной строки, а в отладчике пройдена по шагам: точка останова на `main`, Step Over для обычных строк и Step Into для входа в `add()`; значения `num1 = 12`, `num2 = 30`, `sum = 42` наблюдались в процессе выполнения.

Намеренные ошибки показали разницу между стадиями: отсутствие `#include <stdio.h>` обнаруживает компилятор, а вызов функции без определения — компоновщик (`undefined reference` / `Undefined symbols`). Главный вывод: сборка — это не один шаг, а цепочка «препроцессор → компилятор → компоновщик», и понимание того, на какой стадии возникла ошибка, экономит время при разработке.